

Inigo Quilez :: articles :: budhabrot - 2002



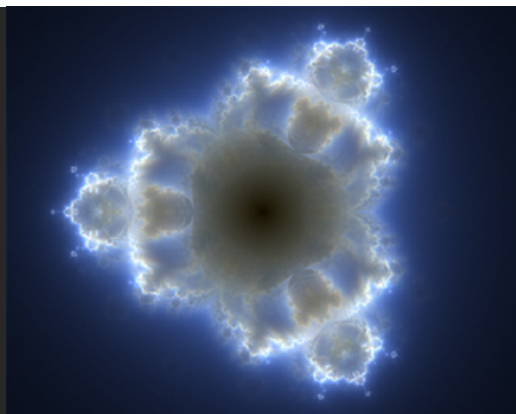
This rendering algorithm was invented by **Melinda Green**. She gives a very concise description of the algorithm, and it is quite straightforward to implement.

The idea is to iterate each point on the complex plane as if you were drawing a standard Mandelbrot set, with a given maximum iteration count limit. Each iterated point will follow an orbit as usual, and what we are going to do is to detect all the escaping orbits and record each point of such orbits in a density map. So, as with the IFS fractals, we initialize a buffer to zero and we increment each pixel of the buffer each time a point of an escaping orbit falls in it. We would ideally want to uniformly sample the complex plane. This means that we don't get the correct image until we completely sample the plane. What you probably want to do is to implement this as a progressive renderer, so that you can get a sense of the final look gets while it gets computed, quite like in a Montecarlo based raytracers.

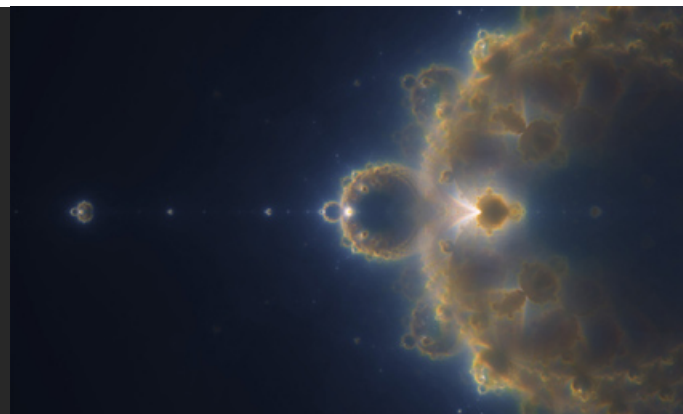
In any case, after a few million iterated points we get our density map ready. We have to play with a few transfer functions as usual to convert this density map into a nice grey level image. I normally use some rooting function (like square or a cubic), and a contrast increasing function after (like a smoothstep()), or something like that).

This of course creates a grey level image. As Melinda explains, a simple trick is to repeat the process two more times with different maximum-iterations limit. That way you get three monochrome images that you can combine into the red, green and blue channels to get your color image.

In its original form, the algorithm allows for computing this three images at once at almost zero cost. Instead of repeating the iterations three times with different iteration-count limit, you just do it once with the maximum of the three limits, and then track the state of the orbit when the iteration count equals the first and second limit.



Cubic Mandelbrot Set. Click to enlarge



Zoom in the standard Mandelbrot Set. Click to enlarge

The software to render these images was as usual very small and simple, I did it in 2006. A C application working without any multithreading or SIMD. The density buffer was done in 32 bit integer per color component, to avoid overflows. That means I used one gigabyte of memory for the 12000x7200 images I rendered.

The conversion from this 96 bit density map to a color image was done in a second step with a different application. This allowed me to tune the transfer functions very quickly without recomputing all the densities each time.

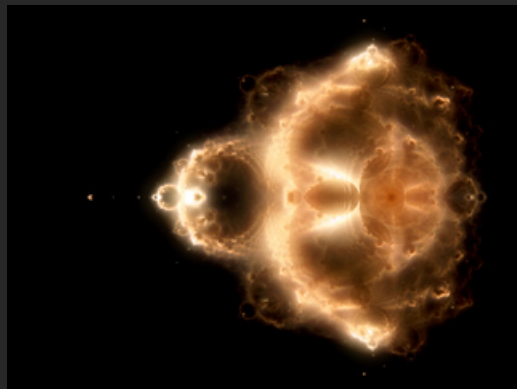
For the images on this page I used random sampling like in Montecarlo methods. I got some problems first with the random number generator, because the 2^{31} period that the standard random (congruential) number generators in the C libraries was not enough to uniformly sample the plane before I got reiterated points. In the Numerical Recipes book I found few 10^{18} period generators (much better!). however even that was not good enough, so I changed the random number generator once more to make sure I didn't end up with a biased image. I used MT19937 generator that has 2^{19937} period (astronomical period as their authors -[Takuji Nishimura and Makoto Matsumoto](#) - say).

Just during the days I made this image a surprisingly interesant post was done in the sci.fractals group. Alex Steckles had the idea of using the Metropolis integration method to the Budhabrot rendering. The algorithm has been successfully used to accelerate Montecarlo based pathtracers, so why not to apply it here also? The method works so fine that you can even zoom on the Mandelbrot set and still get decent images in short time, something impossible before. I tried it but I got biased images, so I guess that I will have to figure out where the bug is...

Based on the same idea, I decided to try a kind of genetic algorithm to choose my sampling points, and indeed I also got dense images for previously loooooong to render regions of the Mandelbrot set. But again, I got very

biased images so I decided not to go in that direction. So I guess that if I even come back to the Budhabrot algorithm, I will have to review the Metropolis code.

In 2002 I also played with budhabrot images, at as low resolution as 2560x1920, what was quite good at that time. For speeding up the calculations I used an ugly trick that I prefer not to mention here, and that caused those artifacts visible in the image (like the color discontinuity in the period-two bulb).



inigo quilez - learning computer graphics since 1994